

게임 트리와 탐색 기반

AI 이해

Game Tree · Minimax · Alpha-Beta Pruning

- 1. 게임 트리(Game Tree) 개념
- 2. 상태(State)와 노드(Node)
- 3. Max / Min 구조
- 4. 분기 계수와 탐색 깊이
- 5. 탐색 공간 폭발과 최적화

게임 트리(Game Tree)의 개념

01 개념 정의

AI는 현재 가능한 수를 모두 살펴보고, 그 수를 두었을 때 상대가 어떤 수를 둘 수 있는지, 다시 그 이후에는 자신이 어떤 선택을 할 수 있는지를 반복적으로 예측합니다.

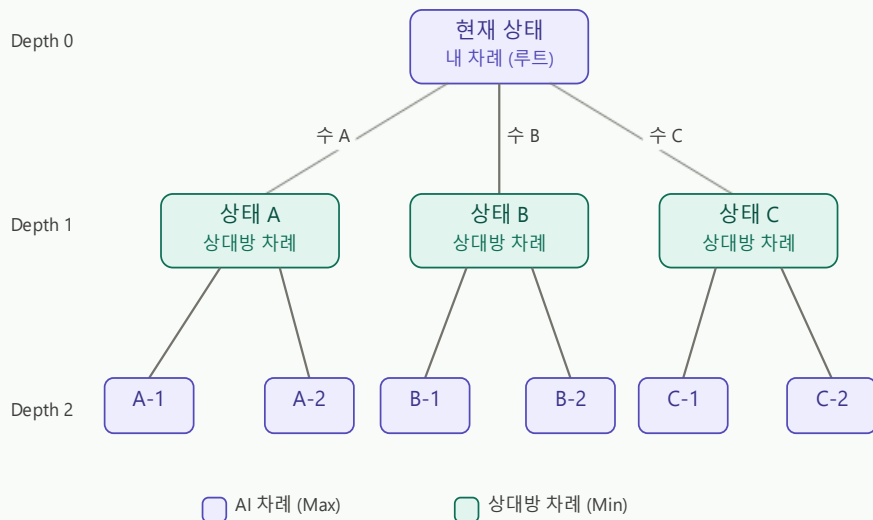
현재 상태에서 가능한 선택들을 나무 구조처럼 펼쳐 놓은 것을 게임 트리(Game Tree)라고 합니다.

체스에서 하나의 수를 둔다는 것은 단순한 이동이 아니라, 새로운 게임 상태를 만드는 일입니다.

체스 AI는 현재 선택만 보는 것이 아니라, 선택 이후의 결과까지 미리 탐색합니다.

📌 이미지 삽입

게임 트리 구조 다이어그램



상태(State)와 노드(Node)

S

상태 (State)

게임의 현재 상황

- 말의 위치 / 각 말이 놓인 위치
- 현재 차례 (누가 둘 차례인지)
- 체크 여부 (킹이 체크 상태인지)
- 캐슬링 가능 여부
- 양파상 가능 여부
- 게임 종료 여부

N

노드 (Node)

게임 트리에서 상태를 표현하는 지점

- 상태 자체라기보다 게임 트리에서 상태를 나타내는 표현 단위
- 어떤 수(Move) 선택 → 새 상태 생성
→ 다음 노드로 연결
- 노드↔노드 연결 이동 = 간선(edge)
- 하나의 수 = 상태 전이(state transition)

정리: 상태(State) → 노드(Node) → 수(Move) = 간선(Edge) → 게임 트리의 기본 구성 요소

상태(State)와 노드(Node) — 시각적 이해

상태(State) 구성 요소

상태 (State)

게임의 현재 상황을 나타내는 정보의 집합



- 말의 위치
각 말이 놓인 위치
- 현재 차례
누가 수를 둘 차례인지
- 체크 여부
킹이 체크 상태인지
- 캐슬링 가능 여부
캐슬링이 가능한지
- 양파상 가능 여부
양파상이 가능한지
- 게임 종료 여부
게임이 끝났는지

이러한 정보를 모두 포함한 것이 하나의 '상태'이다.

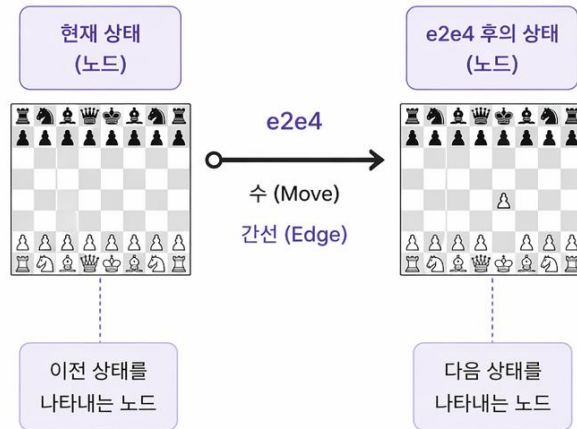


상태를
노드로
표현

노드 → 수(Move) → 새 상태

노드 (Node)

게임 트리에서 하나의 상태를 표현하는 지점



상태 (State)



노드 (Node)



이동 = 간선



새로운 상태

Max / Min 구조

Max Player (AI): 평가값이 가장 높은 수 선택

Min Player (상대): 평가값이 가장 낮은 수 선택

→ 상대는 AI에게 가장 불리한 결과를 유도하는 방향으로 행동한다고 가정합니다.

① Min 단계: 상대의 최선 대응

상대는 각 후보수에서 AI에게 가장 불리한 수 선택

A이후: $\min(4, 2) = 2$ / B이후: $\min(7, 1) = 1$ / C이후: $\min(5, 3) = 3$

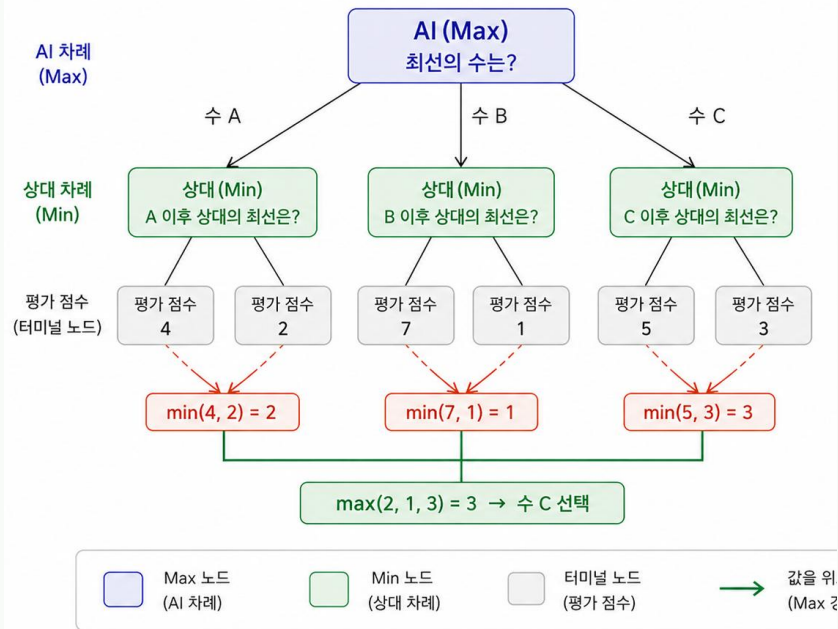
② Max 단계: AI의 최종 선택

AI는 상대의 최선 대응값들 중 가장 큰 값 선택

$\max(2, 1, 3) = 3 \rightarrow$ 수 C 최종 선택

핵심: 즉시 점수가 높은 수가 아닌, 상대 대응 후에도 가장 유리한 수 선택!

Max/Min 트리 다이어그램



Max / Min 구조의 핵심 의미

수 B의 함정



최고 터미널 점수: 7

상대 대응 후 보장값: 1

나쁜 선택 ✗

수 A의 결과



터미널 점수: 4, 2

상대 대응 후 보장값: 2

중간 선택 △

수 C 최선



터미널 점수: 5, 3

상대 대응 후 보장값: 3

최적 선택 ✓

이것이 Max/Min 구조의 핵심이며, 이후 학습할 미니맥스(Minimax) 알고리즘의 기본 원리입니다.

분기 계수(Branching Factor)와 탐색 깊이(Depth)

b 분기 계수 (Branching Factor)

하나의 상태에서 가능한 다음 선택의 개수입니다. 체스에서는 한 상태에서 가능한 합법적 이동이 일반적으로 수십 개 존재합니다.

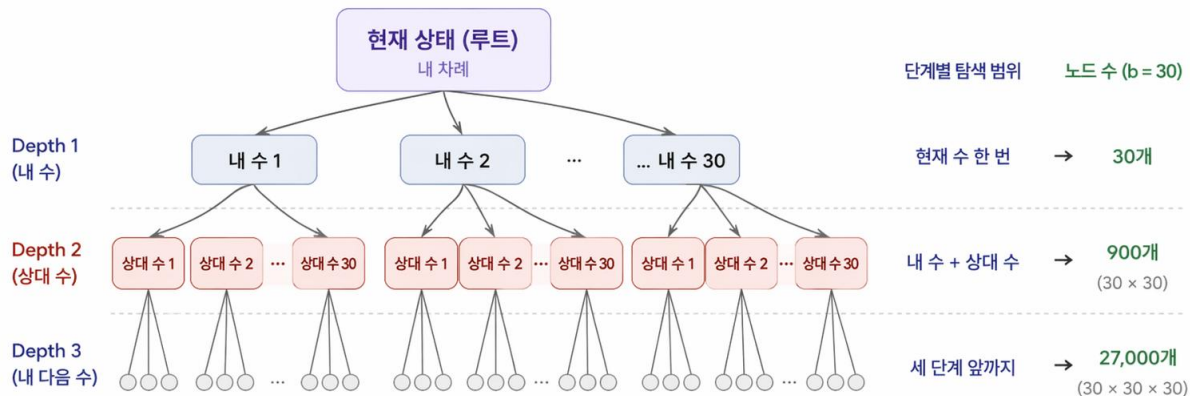
예: 현재 보드에서 평균 30개의 이동이 가능하면 분기 계수 $b = 30$

d 탐색 깊이 (Depth)

AI가 몇 단계 앞까지 미래 상태를 탐색하는지를 나타냅니다. 깊이가 깊어질수록 더 강한 플레이가 가능하지만 계산량이 증가합니다.

예: Depth 3 = 내 수 → 상대 수 → 내 수, 3단계 앞까지 탐색

분기 계수 & 깊이 다이어그램



탐색 상태 수 = b^d (최대) → $O(b^d)$ 시간 복잡도

탐색량의 지수적 증가 (b = 30 기준)

Depth	탐색 범위	차례 구성	탐색 노드 수	수식	지수적 증가
1	현재 수 한 번	내 수	30개	30^1	30
2	내 수 + 상대 수	내 수 → 상대 수	900개	30^2	900
3	세 단계 앞까지	내 수 → 상대 수 → 내 수	27,000개	30^3	27K
4	네 단계 앞까지	내 수 → 상대 수 × 2	810,000개	30^4	810K
5	다섯 단계 앞까지	내 수 → 상대 수 × 2 → 내 수	24,300,000개	30^5	24.3M

깊이가 1씩 증가할 때마다 탐색 노드 수는 30배씩 늘어납니다 → 탐색 공간 폭발(Search Space Explosion)

왜 최적화가 필요한가

문제: 탐색해야 하는 노드 수는 $O(b^d)$ 로 증가합니다. 평균 분기 계수 30인 체스에서 Depth 5까지만 탐색해도 약 2,430만 개의 노드를 처리해야 합니다. '더 깊이 읽을수록 더 강해진다'는 직관은 맞지만, 모든 경우를 완전 탐색하기는 현실적으로 어렵습니다.

해결책: Alpha-Beta Pruning

최종 결과에 영향을 주지 않는 가지를 미리 제거해 불필요한 계산을 줄이는 방법입니다. 같은 시간 안에 더 깊은 탐색을 가능하게 합니다.

학습 로드맵

Max/Min 구조

게임 트리 기본 원리



Minimax 알고리즘

Max/Min을 재귀로 구현



Alpha-Beta Pruning

불필요한 가지 제거

Minimax 알고리즘

이해

재귀 탐색 · 평가 함수 · 결과값 역전파 · 최적 수 결정

- 1. Minimax 기본 원리
- 2. 재귀 함수와 깊이 우선 탐색(DFS)
- 3. 평가 함수(Evaluation Function)
- 4. 결과값 역전파(Backpropagation)
- 5. Minimax 전체 실행 순서

Minimax 알고리즘이란?

핵심 개념

Minimax는 두 플레이어가 번갈아 행동하는 대전형(Adversarial) 게임에서 사용되는 탐색 알고리즘입니다. 상대 또한 최선의 전략을 선택한다고 가정하고, 자신의 최적 수를 계산합니다.

즉, 내 입장에서 가장 좋은 수를 찾되, 동시에 상대가 나에게 가장 불리한 선택을 할 것이라고 가정하는 방식입니다. 이러한 점에서 Minimax를 '상대의 선택까지 고려하는 인공지능 사고 모델'이라 할 수 있습니다.

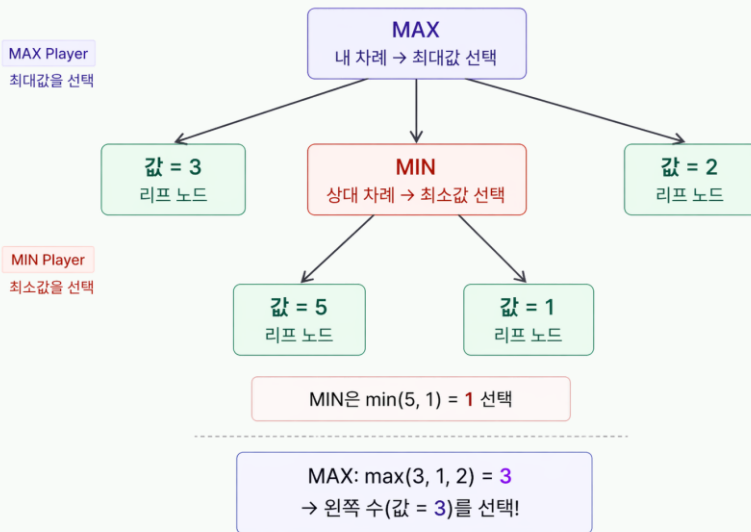
Minimax = "상대가 최악의 대응을 하더라도 가장 덜 손해 보는 선택은 무엇인가?"

단순한 규칙 기반을 넘어, 상대 전략까지 예측하는 게임 AI의 핵심의 사결정 원리

Max Player: 최댓값 선택 (AI)
대)

Min Player: 최솟값 선택 (상

Minimax 기본 트리 다이어그램



Max Player와 Min Player

MAX

Max Player

AI · 최댓값 선택

- 자신의 이익을 최대화하려는 플레이어
- 가능한 선택지 중 가장 높은 평가값의 수 선택
- 일반적으로 인공지능(AI)이 이 역할 수행
- 승리에 가장 가까운 경로를 찾는 것이 목표

예시

A=5, B=8, C=3점일 때
Max Player → B(8) 선택

MIN

Min Player

상대 · 최솟값 선택

- 상대의 이익을 최소화하려는 플레이어
- Max Player에게 가장 불리한 결과를 유도
- 가능한 선택지 중 가장 작은 평가값 선택
- AI의 최악 시나리오를 항상 가정하는 역할

예시

AI가 A=5, B=8, C=3 고려 시
Min Player → C(3) 선택

두 전략이 반복 적용되면서, Minimax는 현재 시점에서 둘 수 있는 수들 중 최적의 수를 결정하게 됩니다.

재귀 함수와 깊이 우선 탐색(DFS)

왜 재귀(Recursive)인가?

Minimax 알고리즘은 재귀 함수(Recursive Function)와 깊이 우선 탐색(DFS) 구조를 기반으로 동작합니다.

재귀 함수란 함수가 자기 자신을 다시 호출하며 문제를 더 작은 단위로 나누어 해결하는 방식입니다. 깊이 우선 탐색은 하나의 경로를 끝까지 탐색한 뒤 다시 이전 단계로 돌아와 다른 경로를 탐색하는 방법입니다. 이러한 구조는 미래의 여러 경우를 예측해야 하는 게임 인공지능 탐색에 매우 적합합니다.

R

재귀 (Recursion)

함수가 자기 자신을 호출하며 문제 분해

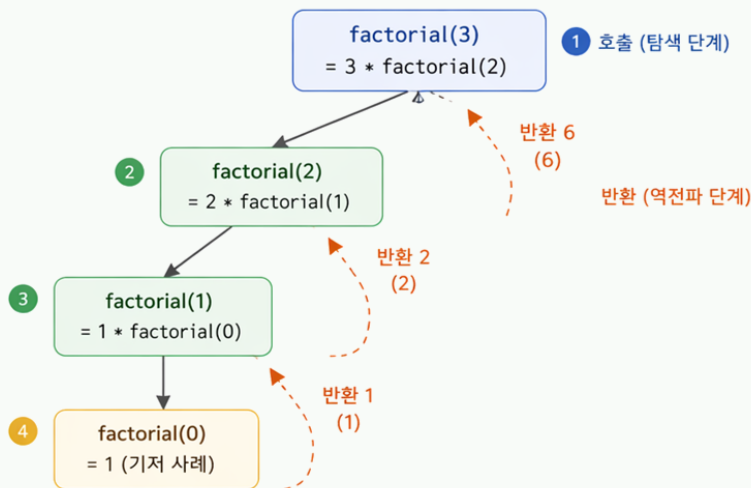
D

DFS

한 경로를 끝까지 탐색 후 복귀

기저 사례(Base Case): 재귀 호출을 종료시키는 조건. Minimax에서는 설정된 깊이에 도달하거나 게임이 종료된 상태가 이에 해당합니다.

재귀 탐색 다이어그램



평가 함수(Evaluation Function)

평가 함수란?

현재 게임 상태를 하나의 점수로 환산하여, 그 상태가 자신에게 유리한지 또는 불리한지를 판단하도록 해주는 함수입니다. 아직 승패가 결정되지 않은 터미널 노드에서 현재 상태를 정량적으로 평가하는 역할을 수행합니다. Minimax 알고리즘은 이러한 평가값을 기반으로 어떤 수가 더 좋은 선택인지 비교하게 됩니다.

체스 말 가치 예시 (Material Value)



퀸

+9



룩

+5



비숍

+3



나이트

+3



폰

+1



킹

0

$$\text{평가값} = \text{내 말 가치 합} - \text{상대 말 가치 합}$$

양수(+)

현재 상태가 자신에게 유리

0에 근접

양측 전력이 균형 상태

음수(-)

현재 상태가 상대에게 유리

평가 함수 계산 예시

평가 함수 계산 4가지 예시



내 말: $9+5+1=15$

상대 말: $5+1=6$

평가값: **+9**



내 말: $5+3=8$

상대 말: $9+1=10$

평가값: **-2**



내 말: $5+3=8$

상대 말: $5+1=6$

평가값: **+2**



내 말: $9=9$

상대 말: $9+5=14$

평가값: **-5**

핵심 역할

평가 함수는 탐색과 의사결정을 연결하는 핵심 기준이 됩니다. 재귀 탐색을 통해 터미널 노드에 도달하면, 평가 함수가 호출되어 해당 노드의 점수를 계산하고, 이후 결과값 역전파의 입력값으로 사용됩니다.

해석 방법

+10 수준

매우 유리한 상태

0 수준

균형 상태

-10 수준

매우 불리한 상태

평가 함수는 단순히 점수를 계산하는 수식이 아니라, 현재 게임 상태의 전략적 우위를 정량적으로 표현하는 도구입니다.

Minimax는 이 평가값을 기반으로 각 터미널 노드를 비교하고, 결과값 역전파를 통해 최적의 수를 선택하게 됩니다.

결과값 역전파(Backpropagation)

역전파 과정이란?

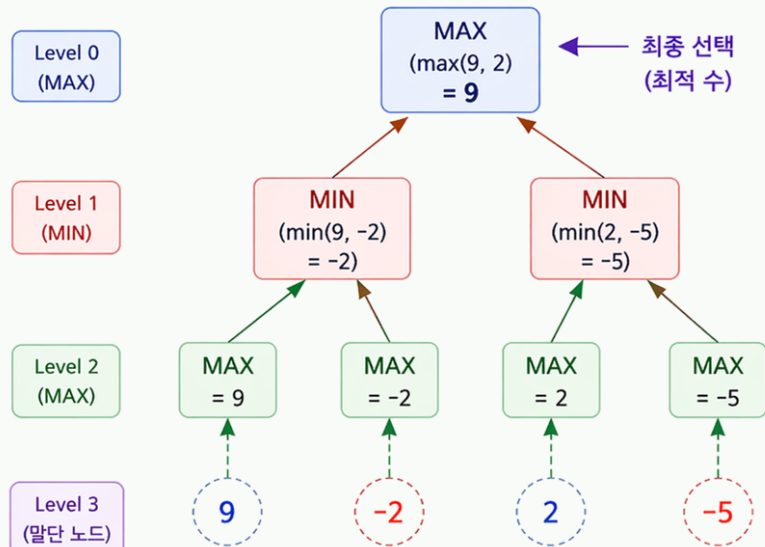
터미널 노드에서 계산된 평가값이 하위 노드에서 상위 노드로 전달되는 과정입니다. 이 과정에서 Max Player와 Min Player의 선택 원리가 순차적으로 적용됩니다.

역전파 과정에서 Min 노드는 자식 노드들 중 최솟값을, Max 노드는 최댓값을 선택하여 부모 노드로 전달합니다. 이 과정이 루트 노드까지 반복되면서 최종적으로 최적의 수가 결정됩니다.

Level 3 (터미널)	9, -2, 2, -5
Level 2 (MAX)	MAX 노드: 값 그대로 전달
Level 1 (MIN)	$\min(9, -2) = -2$ / $\min(2, -5) = -5$
Level 0 (MAX · 루트)	$\max(-2, -5) = -2 \rightarrow$ 최적 수 결정

호출(DFS) → 하강 / 반환(역전파) → 상승 — Minimax의 핵심 메커니즘

결과값 역전파 트리 다이어그램



Minimax 전체 실행 순서 (5단계)

재귀 탐색 (DFS)

1단계

현재 게임 상태에서 가능한 모든 수를 트리 형태로 확장하며 미래 상태를 탐색합니다.



터미널 노드 도달

2단계

설정된 깊이까지 진행하여 터미널 노드에 도달합니다. 더 이상 탐색 대신 현재 상태를 평가하는 단계로 진입합니다.



평가 함수 계산

3단계

터미널 노드의 보드 상태를 점수(평가값)로 변환합니다. 현재 상태의 유리함을 나타내며 이후 의사결정의 기준이 됩니다.

결과값 역전파

4단계

계산된 평가값이 하위 노드에서 상위 노드로 전달됩니다. Min 노드는 최솟값을, Max 노드는 최댓값을 선택하며 상대 최선 대응을 반영합니다.



최적 수 결정

5단계

루트 노드에서 최종적으로 최적의 수가 도출됩니다. Minimax는 최악의 경우에도 가장 유리한 선택을 찾아냅니다.

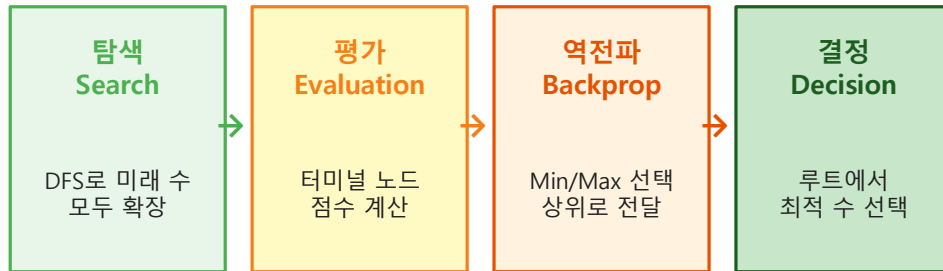
탐색(Search) → 평가(Evaluation) → 역전파(Backpropagation) → 결정(Decision) — Minimax 핵심 작동 원리

Minimax 실행 흐름 정리

5단계 흐름도



핵심 흐름 4단계 요약



Minimax 알고리즘 구현

틱택토 게임으로 배우는 AI 의사결정 알고리즘

3x3 보드 게임을 통해 Minimax 알고리즘의 핵심 구조를 학습하고,
Python으로 AI와 대결하는 틱택토 프로그램을 직접 구현합니다.

X 플레이어 (사람)

O AI (Minimax)

빈 칸 (후보 수)

Minimax 게임 설계

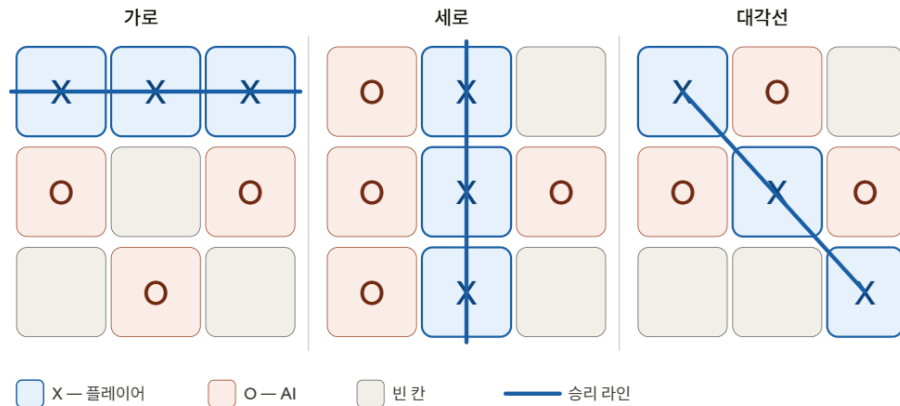
왜 틱택토인가?

체스는 복잡한 규칙과 특수 조건으로 처음부터 Minimax를 적용하기 어렵습니다.

틱택토는 규칙이 단순하지만 미래 상태를 탐색하고 최적의 수를 선택해야 한다는 점에서 Minimax 알고리즘 학습에 최적입니다.

게임 진행 방식

- ① 사람(x)이 먼저 빈 칸을 선택하여 x를 놓습니다
- ② AI가 Minimax 알고리즘으로 최선의 위치에 o를 놓습니다
- ③ 한쪽이 승리하거나 무승부가 될 때까지 반복합니다



보드 구조 설계 & 상태 표현

2. 보드 구조 (1차원 리스트)

```
board = [" "] * 9
```

```
0 | 1 | 2
--+-+--
3 | 4 | 5
--+-+--
6 | 7 | 8
```

예) 사용자가 4를 입력
→ 가운데 칸에 x가 놓임

3. 상태(State) 표현

보드에 놓인 x, o, 빈 칸의 배치 → 하나의 상태(State)

```
board = ["X", " ", "O", " ", "X", " ", " ", " ", "O"]
```

```
  |   |   X |   | O
--+-+--
  | X |   | X |
--+-+--
  |   |   |   | O
```

4. 가능한 수(Legal Moves) 설계

가능한 수 = 현재 비어 있는 칸의 인덱스 목록

```
def legal_moves(b):  
    return [i for i in range(9) if b[i] == " "]
```

예시: 아래 보드에서 가능한 수

x				o
-+--+--				
		x		
-+--+--				
				o

빈 칸: 1, 3, 5, 6, 7 → [1, 3, 5, 6, 7]

이 함수는 현재 상태에서 둘 수 있는 모든 후보 수를 찾습니다.
AI는 이 후보 수들을 하나씩 가정해보고 가장 좋은 수를 선택합니다.

핵심 함수 구현 (1) — winner & legal_moves

```
HUMAN = "X"
AI = "O"
EMPTY = " "
board = [EMPTY] * 9

def print_board():
    print()
    for i in range(3):
        row = board[i * 3:(i + 1) * 3]
        print(" | ".join(row))
        if i < 2:
            print("---+---+---")
    print()

def print_position_guide():
    print("칸 번호 안내")
    print("0 | 1 | 2")
    print("---+---+---")
    print("3 | 4 | 5")
    print("---+---+---")
    print("6 | 7 | 8")
    print()
```

✓ 승리 조건 8가지

가로 3줄, 세로 3줄, 대각선 2줄

✓ 반환값

"X" | "O" → 승자

"DRAW" → 무승부

None → 게임 진행 중

핵심 함수 구현 (2) — minimax & best_ai_move

```
def winner(b):
    win_cases = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]
    for r1, r2, r3 in win_cases:
        if b[r1] == b[r2] == b[r3] and b[r1] != EMPTY:
            return b[r1]
    if EMPTY not in b:
        return "DRAW"
    return None

def legal_moves(b):
    return [i for i in range(9) if b[i] == EMPTY]

def minimax(b, is_maximizing):
    result = winner(b)
    if result == AI:
        return 10
    elif result == HUMAN:
        return -10
    elif result == "DRAW":
        return 0
```

AI 차례 (is_maximizing=True)

모든 가능한 수를 시도
→ 최대 점수(max) 선택

사람 차례 (is_maximizing=False)

모든 가능한 수를 시도
→ 최소 점수(min) 선택

종료 조건

AI 승: +10
사람 승: -10
무승부: 0

play_game() 함수 & VS Code 실행

```
def play_game():
    if is_maximizing:
        best_score = -999
        for move in legal_moves(b):
            b[move] = AI
            score = minimax(b, False)
            b[move] = EMPTY
            best_score = max(best_score, score)
        return best_score
    else:
        best_score = 999
        for move in legal_moves(b):
            b[move] = HUMAN
            score = minimax(b, True)
            b[move] = EMPTY
            best_score = min(best_score, score)
        return best_score

def best_ai_move():
    best_score = -999
    best_move = None

    for move in legal_moves(board):
        board[move] = AI
        score = minimax(board, False)
        board[move] = EMPTY
        if score > best_score:
            best_score = score
            best_move = move
    return best_move

def play_game():
    print("틱택토 Minimax 게임")
    print("플레이어: X / AI: O")
    print_position_guide()
    while True:
        print_board()
        # 사용자 입력
        try:
            move = int(input("X를 놓을 칸 번호를 입력하세요(0~8): "))
        except ValueError:
            print("숫자를 입력하세요.")
            continue
```

틱택토 게임 구현하기

```
if move not in range(9):
    print("0부터 8까지의 숫자만 입력하세요.")
    continue
if board[move] != EMPTY:
    print("이미 말이 있는 칸입니다.")
    continue
board[move] = HUMAN
if winner(board):
    break
# AI 차례
ai_move = best_ai_move()
board[ai_move] = AI
print(f"AI가 {ai_move}번 칸에 0을 놓았습니다.")
if winner(board):
    break
print_board()
result = winner(board)
if result == HUMAN:
    print("플레이어 승리!")
elif result == AI:
    print("AI 승리!")
else:
    print("무승부입니다.")
play_game()
```

VS Code 상단 메뉴에서 터미널을 실행한다.

Terminal → New Terminal

터미널에서 현재 파일이 저장된 폴더로 이동한 뒤 다음 명령어를 입력한다.

python tictactoe_minimax.py

정상적으로 실행되면 다음과 같이 틱택토 게임 안내와 칸 번호가 출력된다.

```
틱택토 Minimax 게임
플레이어: X / AI: O
칸 번호 안내
0 | 1 | 2
---+---+---
3 | 4 | 5
---+---+---
6 | 7 | 8
```

사용자는 0부터 8까지의 숫자 중 하나를 입력하여 원하는 칸에 x를 놓을 수 있다.

(2) 실행 결과 확인

프로그램이 실행되면 사용자가 먼저 x를 놓고, 그다음 AI가 Minimax 알고리즘을 이용해 o를 놓는다. 예를 들어 사용자가 4번 칸을 선택하면 가운데 칸에 x가 놓인다.

```
  |  |
---+---+---
  |  |
---+---+---
  |  |
X를 놓을 칸 번호를 입력하세요 (0~8): 4
```

이후 AI는 가능한 빈 칸을 모두 탐색한 뒤 가장 유리한 위치에 o를 둔다.

```
AI가 0번 칸에 0을 놓았습니다.
0 |  |
---+---+---
  | X |
---+---+---
  |  |
```

승패 판정과 평가 함수 설계

재귀 함수로 **Minimax** 구현

게임 종료 조건 판정 → 평가값 설계 → Minimax 재귀 탐색 → 최적 수 선택 과정을 단계별로 구현합니다

1부

승패 판정과 평가 함수

2부

재귀 함수로 Minimax 구현

3부

최적 수 선택 과정

1. 승리 조건 설계

같은 기호가 가로·세로·대각선 중 하나의 줄에 연속으로 3개 놓이면 승리합니다.

3×3 보드는 0~8 인덱스로 표현되므로, 승리 가능한 8가지 조합을 미리 정리할 수 있습니다.

```
0 | 1 | 2
---+---+---
3 | 4 | 5
---+---+---
6 | 7 | 8
```

```
win_cases = [
    (0, 1, 2), (3, 4, 5), (6, 7, 8), # 가로
    (0, 3, 6), (1, 4, 7), (2, 5, 8), # 세로
    (0, 4, 8), (2, 4, 6)             # 대각선
]
```

```
x |   | o
---+---+---
   | x |
---+---+---
o |   | x
```

2. 게임 종료 조건 & winner() 함수

① x 승리

② o 승리

③ 빈 칸 없음 → 무승부

④ 게임 진행 중 → None

```
def winner(b):  
    win_cases = [  
        (0, 1, 2), (3, 4, 5), (6, 7, 8),  
        (0, 3, 6), (1, 4, 7), (2, 5, 8),  
        (0, 4, 8), (2, 4, 6)  
    ]  
    for r1, r2, r3 in win_cases:  
        if b[r1] == b[r2] == b[r3] and b[r1] != EMPTY:  
            return b[r1]  
    if EMPTY not in b:  
        return "DRAW"  
    return None
```

반환값 정리

"X" → 플레이어 승리

"O" → AI 승리

"DRAW" → 무승부 (빈 칸 없음)

None → 게임 진행 중

3. 평가값(Score) 설계

Minimax는 게임 결과를 문자열이 아닌 점수로 비교합니다.
승리·패배·무승부를 수치로 변환하는 평가값이 필요합니다.

+10

AI (O) 승리

AI에게 가장 유리한 결과

-10

플레이어 (X) 승리

AI에게 가장 불리한 결과

0

무승부

누구에게도 유리하지 않은 결과

```
result = winner(b)
if result == AI:
    return 10
elif result == HUMAN:
    return -10
elif result == "DRAW":
    return 0
```

```
X | O | X
---+---+---
    | O |
---+---+---
X | O |
```

1. 재귀 함수의 기본 구조

Minimax는 단순히 현재 가장 좋아 보이는 수를 고르는 것이 아니라,
가능한 미래 상태를 재귀적으로 탐색하고 결과를 역으로 전달받아 최적의 수를 결정합니다.

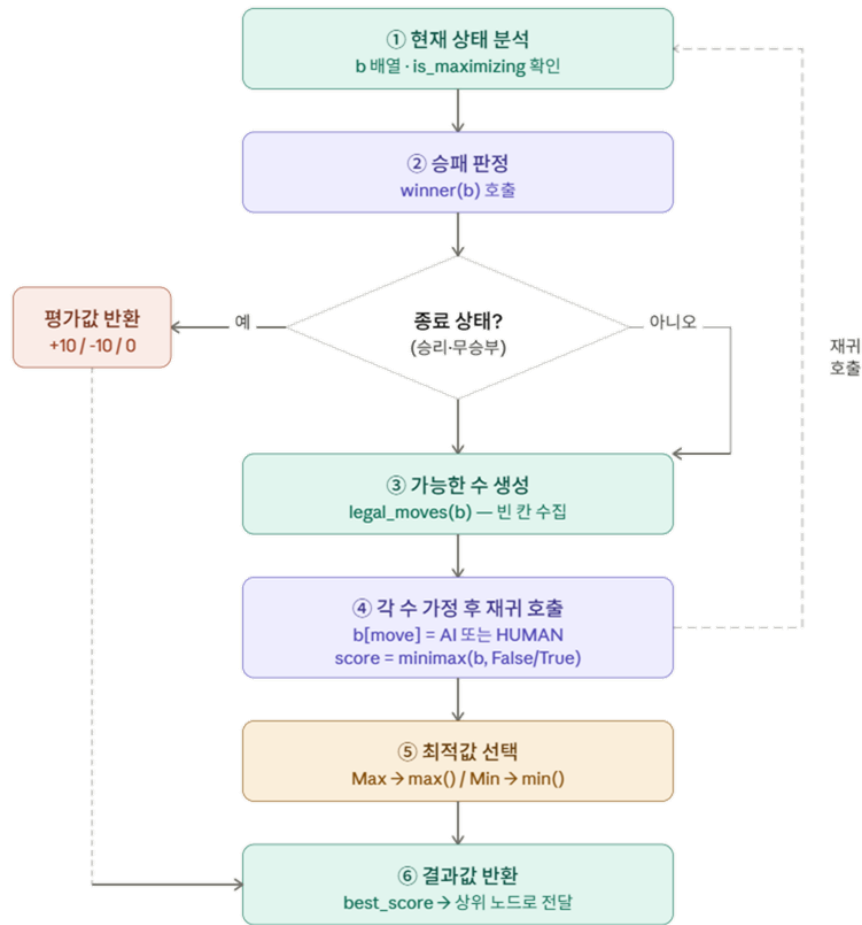
함수 인자

b

현재 게임 상태 (보드 배열)

is_maximizing

True = AI(Max) 차례 / False = 상대(Min) 차례



6단계 재귀 탐색 흐름

1

현재 상태 분석

b 배열·is_maximizing 확인

2

승패 판정

winner(b) 호출 — 종료 상태이면 평가값 즉시 반환

3

가능한 수 생성

legal_moves(b) — 빈 칸 인덱스 수집

4

각 수 가정 후 재귀 호출

b[move]=AI/HUMAN → minimax(b,False/True) → 원상 복구

5

최적값 선택

AI 차례: max() / 상대 차례: min()

6

결과값 반환

best_score → 상위 노드로 전달

2. Max Player & Min Player 구현

(1) Max Player — AI 차례 (is_maximizing=True)

```
best_score = -999
for move in legal_moves(b):
    b[move] = AI
    score = minimax(b, False)
    b[move] = EMPTY
    best_score = max(best_score, score)
return best_score
```

MAX Player (AI)

- best_score = -999 초기화
- 모든 후보 수 시도
- max()로 최댓값 유지
- 최적값 반환

(2) Min Player — 상대 차례 (is_maximizing=False)

```
best_score = 999
for move in legal_moves(b):
    b[move] = HUMAN
    score = minimax(b, True)
    b[move] = EMPTY
    best_score = min(best_score, score)
return best_score
```

MIN Player (상대)

- best_score = 999 초기화
- 모든 후보 수 시도
- min()로 최솟값 유지
- 최적값 반환

(3) 종료 조건 & 반환값 역전파

```
result = winner(b)
if result == AI:
    return 10
elif result == HUMAN:
    return -10
elif result == "DRAW":
    return 0
```

+10 AI 승리 — 탐색 종료, 최대값 전달

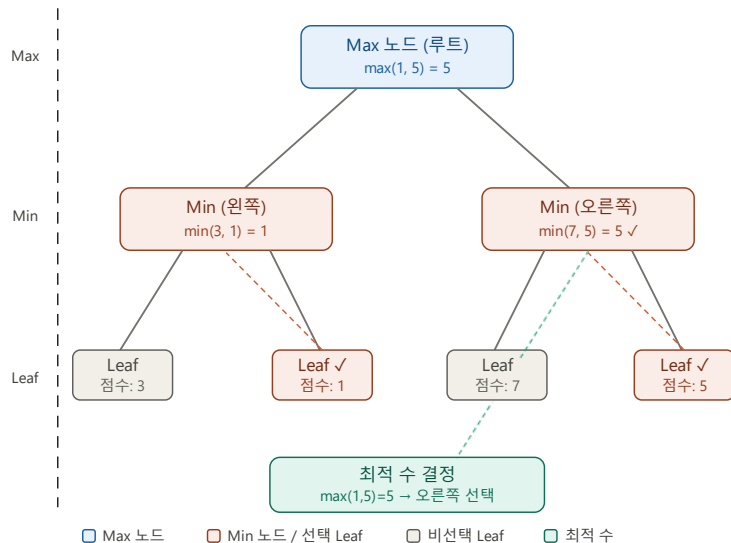
-10 사람 승리 — 탐색 종료, 최소값 전달

0 무승부 — 탐색 종료, 중립값 전달

3. 반환값 역전파 (Backpropagation)

말단 노드(Leaf Node)에서 계산된 평가값은 재귀 호출이 종료되면서 상위 노드로 전달됩니다.

- Min 노드: 전달받은 값 중 최솟값 선택
- Max 노드: 최댓값 선택 → 루트까지 반복하여 최적의 수 결정



4. 최적 수 선택 과정 — `best_ai_move()`

①

가능한 수 가정

`legal_moves(b)` → 빈 칸 순서대로 시도
`b[move] = AI`

②

Minimax 점수 계산

`score = minimax(b, False)`
재귀 탐색 후 점수 반환

③

최적 수 선택

가장 높은 점수의 수 선택
`max(1, 5) = 5` → 수 B 선택

```
def best_ai_move():  
    best_score = -999  
    best_move = None  
    for move in legal_moves(b):  
        b[move] = AI  
        score = minimax(b, False)  
        b[move] = EMPTY  
        if score > best_score:  
            best_score = score  
            best_move = move  
    return best_move
```

동작 요약

- ① 모든 빈 칸 탐색
- ② AI가 O를 둔다고 가정
- ③ `minimax()`로 점수 계산
- ④ 보드를 원상 복구 (백트래킹)
- ⑤ 가장 높은 점수의 칸 저장
- ⑥ `best_move` 반환

백트래킹과 전체 알고리즘 흐름

백트래킹(Backtracking)

AI가 특정 칸에 O를 놓는다고 가정하고 점수를 계산한 뒤,
반드시 board[move] = EMPTY로 원상 복구하여 다음 후보 수 탐색에 영향을 주지 않아야 합니다.

```
board[move] = EMPTY (백트래킹 코드)
```

```
for move in legal_moves(b):  
  
    board[move] = AI  
  
    board[move] = EMPTY  
  
    score = minimax(b, False)
```

전체 알고리즘 구성 요약

함수	역할	반환값
winner(b)	현재 보드 승패 판정	"X" / "O" / "DRAW" / None
legal_moves(b)	가능한 수 목록	[0, 3, 5, ...]
minimax(b, flag)	재귀 탐색으로 점수 계산	-10 ~ +10
best_ai_move()	AI 최적 수 선택	정수 (칸 번호)

Alpha-Beta Pruning

최적화

Minimax의 연산 폭발 문제를 해결하기 위해 Alpha와 Beta 기준값으로 불필요한 탐색 분기를 제거하는 가지치기 알고리즘을 학습합니다.

Minimax의 한계

Alpha / Beta 값의 의미

가지치기(Pruning) 개념

Minimax의 한계 — 연산 폭발(Combinatorial Explosion)

Minimax 시간 복잡도: $O(b^d)$ (b = 분기 인수, d = 탐색 깊이)

탐색 깊이가 한 단계 증가할 때마다 계산량은 분기 인수만큼 곱해져 지수적으로 증가합니다.

체스처럼 분기 인수 ~30인 게임에서 깊이 4만 탐색해도 수십만 개 이상의 상태를 계산해야 합니다.

깊이	노드 수 ($b=5$)
1	5
2	25
3	125
4	625

순수 Minimax만으로는 복잡한 게임의 효율적인 탐색이 불가능 → Alpha-Beta Pruning 등장

(1) Alpha(α) 값의 의미

α (Alpha)

Max Player가 현재까지 탐색한 경로들 중 보장할 수 있는 최선(최대) 점수

초기값: $-\infty$ 으로 시작 $\rightarrow$ 더 좋은 값이 발견될 때마다 갱신

예시: Max 노드 탐색 결과

3 발견 $\rightarrow \alpha = -\infty$

5 발견 $\rightarrow \alpha = 3$

7 발견 $\rightarrow \alpha = 5$

$\alpha = 7$

Alpha = Max의 하한선 $\rightarrow$ "적어도 이 점수는 얻겠다"는 보장값

(2) Beta(β) 값의 의미

β (Beta)

Min Player가 현재까지 탐색한 경로들 중 보장할 수 있는 최선(최소) 점수

초기값: $+\infty$ 으로 시작 $\rightarrow$ 더 작은 값이 발견될 때마다 갱신

예시: Min 노드 탐색 결과

처음 $\beta = +\infty$

8 발견 $\rightarrow \beta = 8$

4 발견 $\rightarrow \beta = 4$

2 발견 $\rightarrow \beta = 2 \leftarrow$ 최종

α Max의 현재 최고값

"적어도 이 점수는 얻겠다"

Max의 하한선

β Min의 현재 최솟값

"이 점수 이상은 허용하지 않겠다"

Min의 상한선

가지치기가 가능한 이유 — $\alpha \geq \beta$ 조건

핵심 조건: $\alpha \geq \beta$ 가 성립하면 해당 분기의 나머지 자식 노드들은 탐색하지 않아도 된다

Max Player는 이미 α 만큼 유리한 선택을 확보한 상태이고, **Min Player**는 β 보다 큰 값을 허용하지 않으려 합니다. $\alpha \geq \beta$ 가 되는 순간 Max는 이미 충분히 좋은 선택을 가지고 있고, Min은 그보다 좋은 결과를 허용하지 않으므로 이후 자식 노드를 더 살펴봐도 현재 선택 결과가 바뀔 수 없습니다.

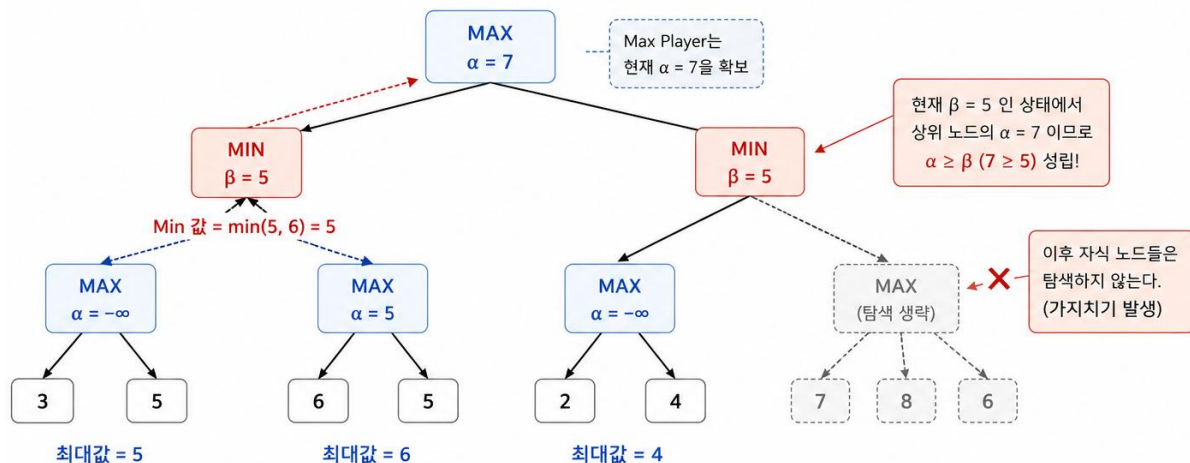
Max의 관점 (α)

이미 α 이상의 결과를 보장받은 상태
→ Min이 더 낮은 값만 줄 수 있다면
결과가 달라질 가능성 없음

Min의 관점 (β)

β 이하의 결과만 상대에게 허용
→ Max가 이미 β 이상을 확보했다면
Min 입장에서 선택할 이유 없음

가지치기 동작 예시



단계별 설명

① Max 노드

$\alpha = -\infty$ 초기화

② Min 탐색

β 갱신 시작

③ α 갱신

더 좋은 값 발견시 갱신

④ $\alpha \geq \beta$ 성립

이후 자식 노드 탐색 중단

⑤ 가지치기

불필요한 분기 제거

예시: Min 노드 $\beta=5$, 상위 Max 노드 $\alpha=7 \rightarrow 7 \geq 5$ 성립 $\rightarrow$ 이후 자식 노드 전체 가지치기

예제 코드 : alphabeta_trace.py

간단한 게임 트리로 Alpha-Beta Pruning이 실제로 어떻게 불필요한 탐색을 제거하는지 확인합니다.

예제 게임 트리

A : [3, 5]
B : [2, 9]
C : [1, 4]

[실행 결과]

최종 선택: A
최종 점수: 3

가지치기 발생 과정: 후보 B 탐색 중 첫 번째 값 2 확인 → $\beta=2$ → 이전 탐색으로 $\alpha=3$ 이미 확보
 $\alpha \geq \beta$ ($3 \geq 2$) 성립 → 나머지 값 9는 탐색 생략 → 계산량 절감!

```
tree = {
    "A": [3, 5],
    "B": [2, 9],
    "C": [1, 4]
}
alpha = -999
best_move = None
print("Alpha-Beta Pruning 탐색 실습")
for move in tree:
    print(f"[{move}] 후보 탐색 시작")
    beta = 999
    for value in tree[move]:
        print(f"  값 {value} 확인")
        beta = min(beta, value)
        print(f"  현재 beta = {beta}")
        if alpha >= beta:
            print("    가지치기 발생!")
            print(f"    alpha({alpha}) >= beta({beta})")
            break
    print(f"{move} 결과값 = {beta}")

if beta > alpha:
    alpha = beta
    best_move = move
print(f"현재 alpha = {alpha}")
print("-" * 30)
print()
print("최종 선택:", best_move)
print("최종 점수:", alpha)
```

(4) 탐색 생략 조건 & 핵심 정리

탐색 생략 조건: $\alpha \geq \beta \rightarrow$ 현재 분기의 나머지 자식 노드들을 더 이상 탐색하지 않는다

이 조건은 현재 Max Player가 이미 확보한 최선의 값(α)이 Min Player가 허용할 수 있는 최대 한계값(β)보다 크거나 같음을 의미합니다.

이후 남은 자식 노드를 더 탐색하더라도 현재 선택 결과보다 더 좋은 결과가 나올 수 없으므로 나머지 경로 탐색을 생략합니다.



탐색은 줄이되 결과는 동일

최종 선택 결과에 영향을 주지 않는
분기만 논리적으로 제거합니다.



효율적인 최적화

최선의 경우 $O(b^{(d/2)})$ 로 줄어들어
훨씬 깊은 탐색이 가능합니다.



가지치기의 핵심

Minimax의 정확성은 유지하면서 불
필요한 계산량을 크게 절감합니다.

체스 엔진 구조 이해

python-chess와 체스 상태 처리

Minimax·Alpha-Beta Pruning을 실제 체스 AI에 적용하기 위한
체스 엔진 구조, python-chess 라이브러리, UCI 이동 표기법을 단계별로 학습합니다.

체스 엔진 구조

python-chess 설치 & 기능

UCI 이동 표기법 & 좌표

체스 엔진(Chess Engine) 구조

체스 엔진은 현재 보드 상태를 관리하고, 가능한 수를 생성한 뒤 탐색 알고리즘으로 최적의 수를 계산합니다. 단순히 하나의 알고리즘만 사용하는 것이 아니라 여러 기능이 하나의 시스템으로 연결되어 동작합니다.



게임 상태 관리

보드 표현, 합법수 생성
현재 상태 저장·갱신



탐색 알고리즘

Minimax + Alpha-Beta
Quiescence 탐색



평가 함수

기물 가치, 위치 점수
강안전도 수치화



UCI/GUI 인터페이스

외부 프로그램 연결
표준 통신 프로토콜

동작 흐름

- ① 현재 보드 상태 저장
- ② 합법적 수(legal moves) 생성
- ③ Minimax + Alpha-Beta로 게임 트리 탐색
- ④ 평가 함수로 점수 계산 → 최적의 수 출력

python-chess 라이브러리 이해

python-chess는 체스 상태 처리를 파이썬에서 쉽게 구현할 수 있도록 제공되는 오픈소스 라이브러리입니다.

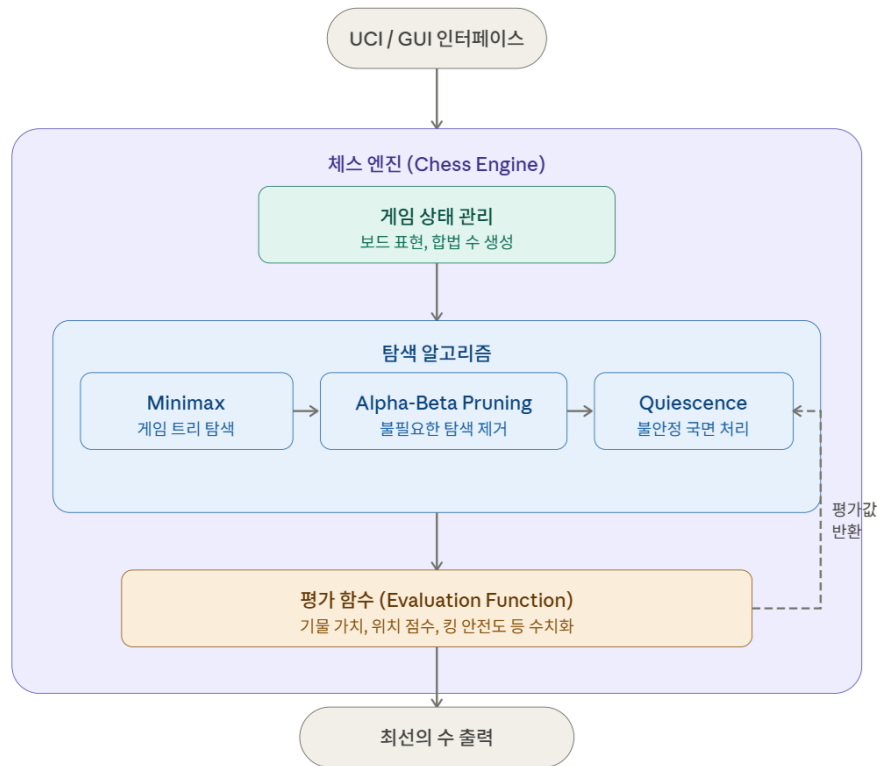
♟ 체스 보드(Board) 생성 및 상태 관리

↔ 말의 이동(Move) 처리

✅ 합법적 수(legal moves) 생성

♔ 게임 종료 판정 (Checkmate, Stalemate 등)

🔗 체스 엔진 및 AI 알고리즘 연결



python-chess 설치하기

1

VS Code에서 프로젝트 폴더 열기

chess_ai_project 폴더 생성 후 VS Code
에서 열기
Terminal → New Terminal

2

Python 설치 확인하기

python --version
→ Python 3.8 이상 확인

3

python-chess 설치하기

pip install chess
또는 pip install python-chess

4

설치 확인

Successfully installed chess-...
→ 설치 완료 메시지 확인

\$ pip install chess

또는

\$ python -m pip install chess



VS Code 설치 완료 메시지
터미널 결과 이미지 삽입

설치 확인 코드 — check_chess.py

```
import chess
board = chess.Board()

print("python-chess 설치 확인 완료")
print()
print("초기 체스 보드:")
print(board)
print()
print("현재 차례:")
print("백" if board.turn == chess.WHITE else "흑")
print()
print("합법적인 수의 개수:")
print(len(list(board.legal_moves)))
```

초기 체스 보드:

```
r n b q k b n r
p p p p p p p p
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
P P P P P P P P
R N B Q K B N R
```

현재 차례:

백

합법적인 수의 개수:

20

`import chess`

python-chess 불러오기 (보드·이동·판정 포함)

`chess.Board()`

초기 체스 보드 생성 (백부터 시작)

`board.turn`

True=백 차례, False=흑 차례

`board.legal_moves`

현재 상태의 모든 합법적 수 객체

가능한 수(legal moves) 확인하기

체스에서는 현재 상태(State)에 따라 이동 가능한 수가 달라집니다.

python-chess는 현재 상태에서 가능한 모든 합법적 이동(legal move)을 객체로 제공합니다.

```
import chess

board = chess.Board()

print("현재 가능한 수:")

for move in board.legal_moves:
    print(move)
```

초기 상태 합법적 수 분석

폰(Pawn) 8개	→ 각 2개 이동 가능 = 16개
나이트(Knight) 2개	→ 각 2개 이동 가능 = 4개
합계	→ 총 20개의 합법적 수

g1h3
g1f3
b1c3
b1a3
h2h3
g2g3
f2f3
e2e3
d2d3
c2c3
b2b3
a2a3
h2h4
g2g4
f2f4
e2e4
d2d4
c2c4
b2b4
a2a4

UCI 이동 표기법의 구조

UCI (Universal Chess Interface) : 체스 엔진과 프로그램이 서로 이동 정보를 주고받기 위한 표준 형식

표기 형식: 출발 좌표 + 도착 좌표

예시: "e2e4" → e2 위치의 말을 e4 위치로 이동

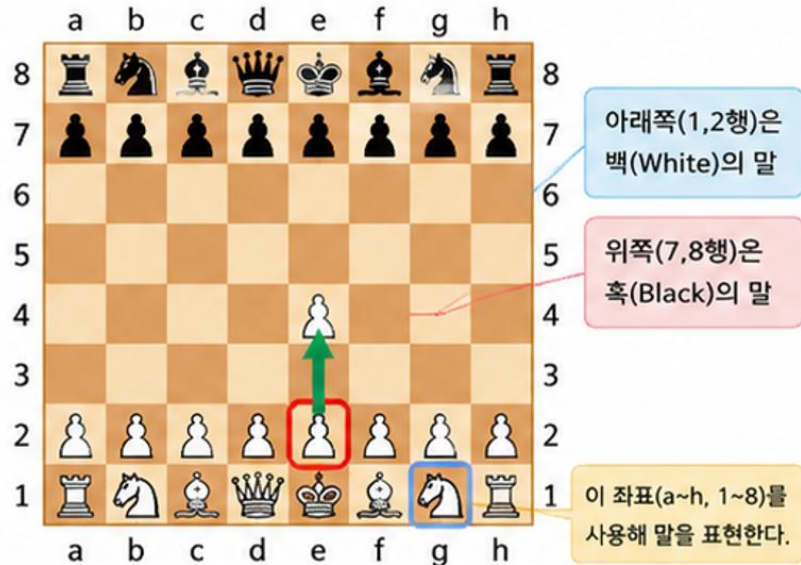
체스 좌표 체계

가로(File): a b c d e f g h

세로(Rank): 1 2 3 4 5 6 7 8

아래쪽(1,2행) = 백(White) / 위쪽(7,8행) = 흑(Black)

이동 코드	의미
e2e4	e2 → e4 (폰 이동)
g1f3	g1 → f3 (나이트 이동)
b1c3	b1 → c3 (나이트 이동)
a2a4	a2 → a4 (폰 이동)



(3) 체스 말 기호 & UCI 이동 코드 예시

P

Pawn
(폰)

N

Knight
(나이트)

B

Bishop
(비숍)

R

Rook
(룩)

Q

Queen
(퀸)

K

King
(킹)

UCI 이동 코드 예시

UCI 표기	의미	말 종류
e2e4	e2 → e4	폰(Pawn)
d2d4	d2 → d4	폰(Pawn)
g1f3	g1 → f3	나이트(Knight)
b1c3	b1 → c3	나이트(Knight)
e1g1	e1 → g1	킹(King) 캐슬링

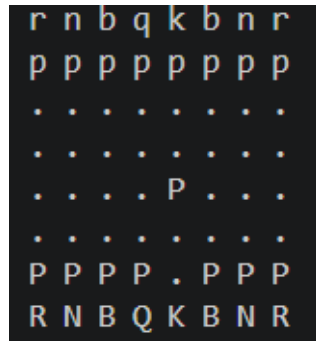
체스판 좌표

8	a8	b8	c8	d8	e8	f8	g8	h8
7	a7	b7	c7	d7	e7	f7	g7	h7
6	a6	b6	c6	d6	e6	f6	g6	h6
5	a5	b5	c5	d5	e5	f5	g5	h5
4	a4	b4	c4	d4	e4	f4	g4	h4
3	a3	b3	c3	d3	e3	f3	g3	h3
2	a2	b2	c2	d2	e2	f2	g2	h2
1	a1	b1	c1	d1	e1	f1	g1	h1
	a	b	c	d	e	f	g	h

UCI 이동 코드 작성 — board.push()

```
import chess

board = chess.Board()
move = chess.Move.from_uci("e2e4")
board.push(move)
print(board)
```



r	n	b	q	k	b	n	r
p	p	p	p	p	p	p	p
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	P	.	.	.
.	.	.	.	.	.	.	.
P	P	P	P	.	P	P	P
R	N	B	Q	K	B	N	R

`chess.Move.from_uci("e2e4")` → 문자열 이동 정보를 Move 객체로 변환

board.push(move) 실행 후 변경 사항:

✓ e2의 폰을 e4로 이동

✓ 현재 차례가 백 → 흑으로 변경

✓ legal moves 목록이 다음 상태 기준으로 갱신

✓ 현재 State → Next State로 전환

체스 상태 처리 & 게임 종료 판정

State 변화 흐름, 게임 종료 판정 함수, 콘솔 기반 체스 프로그램을 단계별로 구현합니다.

상태(State) 변화 과정 정리

게임 종료 판정

콘솔 기반 체스 프로그램

상태(State) 변화 과정 정리

체스 AI는 현재 상태(State)에서 가능한 수(legal moves)를 생성한 뒤,
선택한 수(move)를 적용하여 다음 상태(Next State)로 이동합니다.

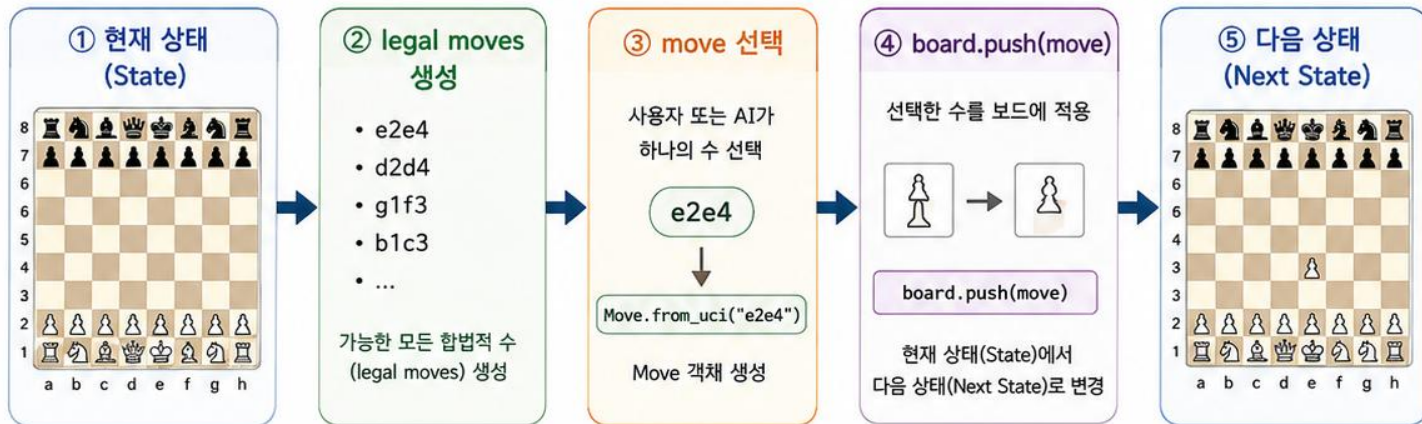
```
chess.Move.from_uci( )
```

문자열 이동 정보를 Move 객체로 변환

```
board.push(move)
```

선택한 수를 현재 보드에 적용 → Next State

상태 변화 과정



게임 종료 판정 함수

`board.is_game_over( )`

게임 종료 여부

True = 종료 / False = 진행 중

`board.is_checkmate( )`

체크메이트 여부

상대 킹이 공격받고 피할 수 없는
상태

`board.result( )`

게임 결과 반환

1-0 / 0-1 / 1/2-1/2 / *

```
import chess
```

```
board = chess.Board()
```

```
board.push(chess.Move.from_uci("f2f3"))
```

```
board.push(chess.Move.from_uci("e7e5"))
```

```
board.push(chess.Move.from_uci("g2g4"))
```

```
board.push(chess.Move.from_uci("d8h4"))
```

```
print(board)
```

```
print("게임 종료 여부:", board.is_game_over())
```

```
print("체크메이트 여부:", board.is_checkmate())
```

```
print("게임 결과:", board.result())
```

```
r n b . k b n r
p p p p . p p p
. . . . . . . .
. . . . p . . .
. . . . . . P q
. . . . . P . .
P P P P P . . P
R N B Q K B N R
게임 종료 여부: True
체크메이트 여부: True
게임 결과: 0-1
```


게임 결과 & 스테일메이트(Stalemate)

(4) board.result() 반환값

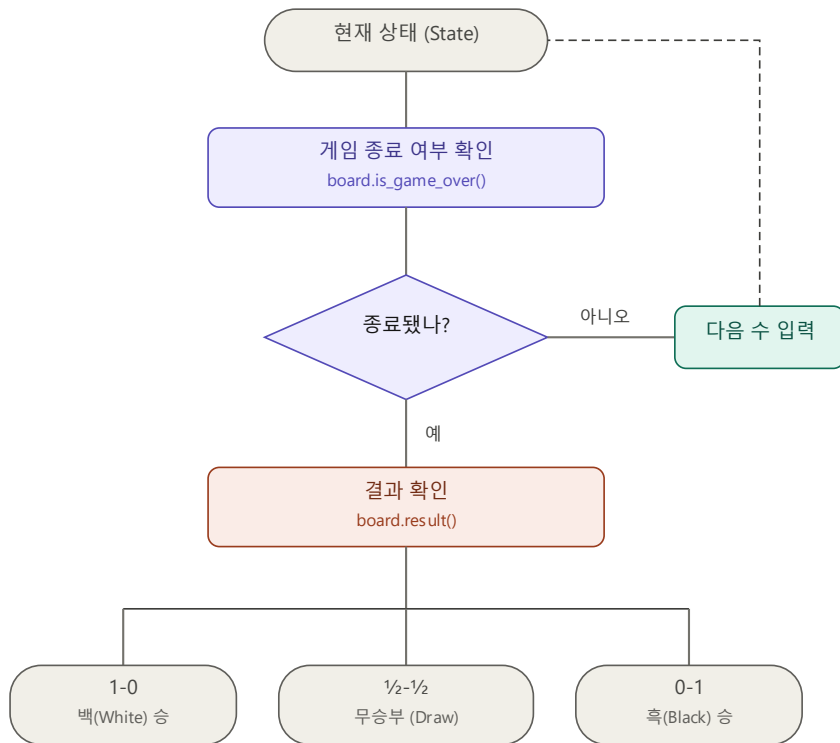
결과	의미
1-0	백(White) 승리
0-1	흑(Black) 승리
1/2-1/2	무승부(Draw)
*	아직 게임 진행 중

(3) 스테일메이트(Stalemate)

킹이 체크 상태는 아니지만 현재 차례에 둘 수 있는 합법적인 수가 없는 상태.

→ 게임은 무승부로 종료됩니다.

→ python-chess는 스테일메이트도 board.is_game_over()에서 자동 판정합니다.



게임 종료 판정 — 종합 코드

체스 AI는 현재 State를 계속 분석하면서 게임 종료 여부를 반복적으로 확인하고, 게임이 끝나면 `board.result()`를 통해 최종 결과를 반환합니다.

```
import chess
```

```
board = chess.Board()
```

```
print("현재 결과:", board.result())
```

현재 결과 : *

```
is_game_over() = False
```

→ 다음 수(move)를 입력받아 게임 계속 진행

```
is_game_over() = True
```

→ `board.result()`로 최종 결과 확인 및 출력

콘솔 기반 체스 프로그램 설계

지금까지 학습한 내용을 이용하면 콘솔(Console) 환경에서 직접 체스를 진행하는 프로그램을 만들 수 있습니다.



사용자 입력 받기



합법적 수(legal moves)
검사



체스 상태(State) 변경



게임 종료 여부 확인



게임 종료까지 반복 실행

핵심 반복 구조

```
while not board.is_game_over():
```

```
    수 입력 → Move 객체 변환
```

```
    합법적 수인지 확인
```

```
    board.push(move) 적용
```

```
print(board.result())
```

콘솔 기반 체스 프로그램 코드

```
import chess
```

```
board = chess.Board()  
print("체스 게임 시작")
```

```
while not board.is_game_over():  
    print("\n현재 보드:")  
    print(board)  
    move = input("수 입력 (예: e2e4): ")  
    try:  
        move = chess.Move.from_uci(move)  
        if move in board.legal_moves:  
            board.push(move)  
        else:  
            print("불가능한 수입니다.")  
    except:  
        print("입력 오류")  
print("게임 종료")  
print("결과:", board.result())
```

```
현재 보드:  
r n b q k b n r  
p p p p p p p p  
.  
.  
.  
.  
.  
.  
.  
P P P P P P P P  
R N B Q K B N R  
수 입력 (예: e2e4): f2f3
```

```
현재 보드:  
r n b q k b n r  
p p p p p p p p  
.  
.  
.  
.  
.  
.  
.  
P P P P P P P P  
R N B Q K B N R  
수 입력 (예: e2e4): e7e5
```

```
현재 보드:  
r n b q k b n r  
p p p p . p p p  
.  
.  
.  
.  
.  
.  
.  
P P P P P P P P  
R N B Q K B N R  
수 입력 (예: e2e4): g2g4
```

```
현재 보드:  
r n b q k b n r  
p p p p . p p p  
.  
.  
.  
.  
.  
.  
.  
P P P P P P P P  
R N B Q K B N R  
수 입력 (예: e2e4): d8h4  
게임 종료  
결과: 0-1
```

핵심: while not board.is_game_over() → 게임이 종료되지 않은 동안 반복 실행

코드 핵심 설명

`try-except` : 프로그램 실행 중 발생할 수 있는 오류(Exception)를 처리하는 구문

사용자가 잘못된 형식의 문자열(abcd, hello, 1234 등)을 입력하면 오류가 발생할 수 있습니다. try 영역에서는 입력된 문자열을 Move 객체로 변환하고, 합법적인 수인지 검사합니다.

합법적 수 검사

```
if move in board.legal_moves:
    board.push(move)
else:
    print("불가능한 수입니다.")
```

스테일메이트(Stalemate)

킹 체크는 아니지만 합법적 수 없음 → 무승부
`board.is_game_over()`에서 자동 판정

오류 처리

```
try:
    move = chess.Move.from_uci(move)
    ...
except:
    print("입력 오류")
```

게임 결과 반환값

1-0 (백 승) / 0-1 (흑 승) / 1/2-1/2 (무승부)
* (아직 진행 중)